

Technical Sheet

Task Management & Analytics Dashboard — Assignment # 3 · Web Technologies (AIT-232T)
Sir Syed University of Engineering & Technology · Spring 2026 · Batch BIAT-2025S

1. API Test Case Execution Checklist

Each endpoint was exercised against a running instance of the Express server with all request/response cycles verified end-to-end (route handler → Mongoose query → database → JSON response). Results below are from the actual test run.

#	Method	Endpoint	Test case	Expected result	Status
1	GET	/api/tasks	Fetch list with no records	200 OK, empty array, count: 0	PASS
2	POST	/api/tasks	Create task with valid title, priority, dueDate	201/200, returns created document with _id	PASS
3	POST	/api/tasks	Submit with title missing	400 Bad Request, validation error message	PASS
4	GET	/api/tasks	Fetch list after one insert	200 OK, count: 1, document present	PASS
5	PATCH	/api/tasks/:id	Toggle completed: true on existing task	200 OK, returns updated document	PASS
6	DELETE	/api/tasks/:id	Delete an existing task	200 OK, success: true	PASS
7	DELETE	/api/tasks/:id	Delete a task that no longer exists	404 Not Found, error message	PASS
8	GET	/api/unknown	Request an undefined route	404 Not Found (Express default handler)	PASS

All 8 test cases passed against the live route handlers prior to deployment.

2. Technical Process Write-Up

2.1 Lifecycle of an AJAX request, from user action to database retrieval

The dashboard never reloads the page; every data operation travels through the same asynchronous round trip:

1. User interaction. A DOM event fires — submitting the add-task form, clicking the complete toggle, or clicking delete — and the corresponding JavaScript event listener runs.

2. Client-side request. The handler builds a payload object and calls the Fetch API (`fetch(url, {method, headers, body})`). For writes, the body is `JSON.stringify()`'d and the Content-Type header is set to `application/json`. The call is non-blocking: the browser keeps rendering while it waits for a response.

3. Network transport. The browser sends an HTTP request over TCP to the Express server. CORS headers (configured via the cors middleware) allow the request when frontend and backend are served from different origins.

4. Server middleware. Express's `express.json()` middleware parses the raw request body into `req.body` before any route handler runs.

5. Routing. Express matches the verb and path (e.g. `POST /api/tasks`) to the corresponding handler in `routes/tasks.js`.

6. Validation. The handler checks required fields (e.g. a non-empty title) before touching the database, returning a 400 response early if validation fails.

7. Database query. Mongoose translates the call (`Task.create()`, `Task.find().sort()`, `findByIdAndUpdate()`, `findByIdAndDelete()`) into a MongoDB wire-protocol command, sent over the persistent connection opened when the server started.

8. Database execution. MongoDB performs the read/write against the tasks collection and returns the resulting document(s) (applying the schema's validation and defaults on insert).

9. Server response. The handler wraps the result in a consistent JSON envelope (`{success, data}`) and sends it back with an appropriate status code (200, 201, 400, or 404).

10. Client-side update. The `fetch()` promise resolves; `response.json()` parses the body; the in-memory tasks array is updated; and a render function rebuilds only the affected DOM nodes (task cards, counters, the completion gauge) — without a full page reload.

2.2 Configuration dependencies managed during production deployment

Environment variables. `MONGO_URI` and `PORT` are never hard-coded. Locally they live in a git-ignored `.env` file loaded by `dotenv`; on the host (Render/Vercel/Heroku) they are set as dashboard secrets so the same codebase runs unmodified in both environments.

Database network access. MongoDB Atlas requires the deployment platform's outbound IPs to be allow-listed (or `0.0.0.0/0` for simplicity), otherwise connection attempts time out even with a correct URI.

CORS policy. The cors middleware must permit the frontend's deployed origin; an overly strict or missing CORS config is a common cause of requests failing only in production.

Build and start commands. The host needs an explicit build step (`npm install`) and start command (`npm start`, which runs `node server.js`) — without these the platform doesn't know how to boot the app.

Static asset serving. Express serves the `frontend/` folder directly (`express.static`), so frontend and backend ship as a single deployable service rather than two separately hosted pieces that would need their own CORS and routing coordination.

Port binding. The server must listen on `process.env.PORT` rather than a fixed port, since the hosting platform assigns the port dynamically at runtime.

Process persistence. Because the app holds an open Mongoose connection rather than connecting per-request, it requires a long-running Node process (a 'web service') rather than a serverless/ function-per-request model.

GitHub Repository: _____ Live URL: _____